

"In a network environment, devices may communicate with a server, but not every device should automatically be trusted. Also, even an authorized device may sometimes generate unusual amounts of traffic. So we wanted to build a system that can identify unauthorized devices and detect abnormal communication behavior."

1. Tomorrow, do you need to log in again?

Yes, probably — but that's completely normal.

Your login is stored in the **database**, so your account does **not** disappear when you close the server.

Tomorrow:

1. Open VS Code.
2. Open the NETGUARD folder.
3. Start the server:

`node server.js`

4. You should see:
5. NETGUARD database connected.
6. Users table ready.
7. Network activity table ready.
8. Devices table ready.

NETGUARD server running on port 3000

9. Open:

`http://localhost:3000`

10. Log in using the account you already created.
11. Dashboard opens.

You do NOT need to create the account again.

2. Do you need to generate a new mobile token?

No.

The mobile device you registered is already stored in the devices table.

Your token is stored in the database.

So tomorrow you should **use the same token**.

However, because the token is long and annoying to type, I strongly recommend we do **one small preparation tonight**:

Save the mobile token somewhere safe.

For example, Notepad:

NETGUARD DEMO PHONE TOKEN

<your actual token here>

Then tomorrow you simply copy → paste it into the phone.

Don't generate another device unless something goes wrong.

3. The most important thing: Understand the whole project

Forget the code for a moment.

Imagine NETGUARD as a **security guard standing at the entrance of a building**.

Your building is the **NETGUARD server**.

Your devices are people trying to enter.

NETGUARD — THE STORY

Imagine your HOD asks:

"How do you know whether a device communicating with your system is trustworthy?"

That's where your project begins.

You can say:

"In a network environment, devices may communicate with a server, but not every device should automatically be trusted. Also, even an authorized device may sometimes generate unusual amounts of traffic. So we wanted to build a system that can identify unauthorized devices and detect abnormal communication behavior."

Then introduce:

"This is NETGUARD — a Network Security Monitoring System."

4. What exactly does NETGUARD do?

NETGUARD has **two main security checks**.

Check 1 — Is the device authorized?

It asks:

"Do I know this device?"

If yes:

 **AUTHORIZED**

If no:

 **UNAUTHORIZED**

Check 2 — Is the authorized device behaving normally?

Even if I know the device, I shouldn't blindly trust it.

NETGUARD watches its request pattern.

If it suddenly sends too many requests within a short period:

 **SUSPICIOUS NETWORK BEHAVIOR**

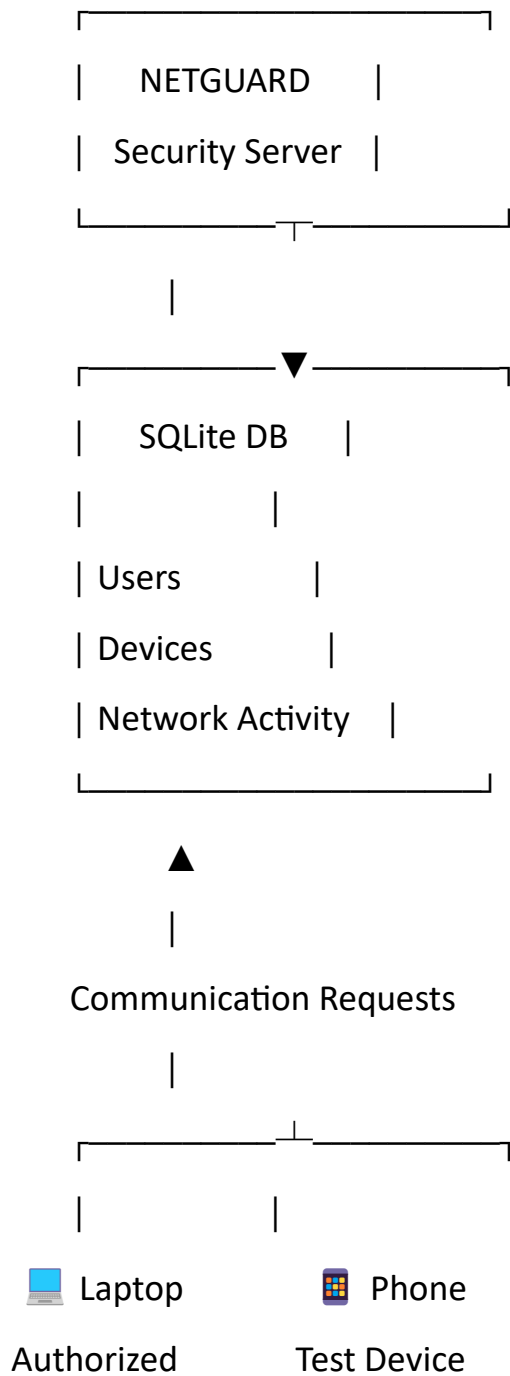
So your project has a nice security concept:

Identity + Behavior

That's a strong way to explain your project.

5. Now understand the complete architecture

You have basically:



Your laptop is running the **Node.js + Express server**.

SQLite stores the information.

The phone communicates with that server.

6. What happens when you log in?

You open:

http://localhost:3000

You see:

NETGUARD

Email

Password

Sign In

You enter your account.

The browser sends:

Email + Password



Server



SQLite

The server checks whether that user exists.

If correct:

Login successful



Dashboard

So login is mainly for accessing the NETGUARD monitoring dashboard.

7. What is SQLite doing?

This is important for viva.

SQLite is your **database**.

Think of it as NETGUARD's memory.

It remembers:

Users

Who can access NETGUARD?

Devices

Which devices are authorized?

Network Activity

What communication happened?

So when you restart the server tomorrow, your data is still there.

That's why:

You don't have to register everything again.

8. Now comes the interesting part — the phone

Your phone is acting as a **controlled test device**.

It connects to the same network as your laptop.

For example:

Phone

|

| Wi-Fi / Hotspot

|



Laptop

NETGUARD Server

Then your phone opens:

<http://LAPTOP-IP:3000/device-test.html>

This page allows the phone to send a controlled communication request to your server.

9. What is the token?

This is probably one of the most important things you should understand.

When you register a device, NETGUARD creates a unique:

Device Token

Think of it like a **digital ID card**.

For example:

PHONE



Device Token



NETGUARD

When the phone sends a request:

"Hello NETGUARD, here is my device token."

The server checks its database.

10. Scenario 1 — Authorized device

You paste your real phone token.

Phone sends:

POST /api/device-communication

with:

device_token = YOUR_REAL_TOKEN

Server checks:

Does this token exist?

|

|— YES



Device authorized

Then NETGUARD records:

AUTHORIZED

Dashboard shows something like:

 **Authorized Device**

You can tell your HOD:

"Here we can see that the registered device has been successfully authenticated and its communication has been allowed."

11. Scenario 2 — Unauthorized device

Now comes your best demo moment.

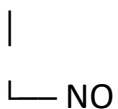
You deliberately enter:

FAKE-DEVICE-TOKEN

The phone sends the request.

NETGUARD checks:

Does this token exist?



Unauthorized Device



BLOCK REQUEST



CREATE ALERT

Dashboard becomes:

● UNAUTHORIZED

And you can say:

"Now I'm simulating an unknown device. The token does not exist in our authorized-device database. NETGUARD therefore rejects the request and records it as a security alert."

That is a very clean demonstration.

12. Scenario 3 — Suspicious behavior

This is actually the coolest part.

Suppose the device **is authorized**.

Normally:

Phone



Request



NETGUARD



AUTHORIZED

That's fine.

But imagine the same device suddenly starts sending many requests extremely quickly.

NETGUARD notices:

Authorized device



Many requests



Short time period



Abnormal pattern



SUSPICIOUS

Your current demo threshold is approximately:

10 authorized requests within 5 seconds.

So after rapidly sending requests, NETGUARD detects:

● **Suspicious Network Behavior**

And the dashboard displays the alert.

Your explanation:

"Notice that the device itself is authorized. So NETGUARD does not classify it as unauthorized. Instead, it analyzes the request pattern and detects that the device is behaving abnormally."

That distinction is **very important**.

13. Your project therefore has THREE states

Remember this tomorrow:

● **NORMAL**

Known device + normal communication.

● **UNAUTHORIZED**

Unknown device.

● **SUSPICIOUS**

Known device + abnormal request behavior.

You can literally remember:

Who are you? → Are you allowed? → Are you behaving normally?

That's the entire project.

14. What does the dashboard show?

Your dashboard gives the security guard's view.

Protected Devices

How many devices are registered.

Network Requests

Communication requests observed by NETGUARD.

Security Alerts

Detected unauthorized/suspicious events.

System Status

Whether a security event has been detected.

Then:

Live Network Activity

Shows communication events.

For example:

Authorized Device

POST /api/device-communication

10.x.x.x

AUTHORIZED

or:

Unauthorized Device

POST /api/device-communication

10.x.x.x

UNAUTHORIZED

or:

Suspicious Network Behavior

POST /api/device-communication

10.x.x.x

SUSPICIOUS

15. What is happening behind the scenes?

This is your technical story.

You have:

HTML



CSS



JavaScript



Express Server



SQLite Database

HTML

Creates the webpage.

CSS

Makes it look professional.

JavaScript

Makes the webpage communicate with the server.

Node.js + Express

Runs your backend/server.

SQLite

Stores users, devices and network activity.

So if your HOD asks:

"What technologies did you use?"

Say:

"We used HTML, CSS and JavaScript for the frontend, Node.js with Express for the backend, and SQLite for persistent data storage."

16. And what does the backend actually do?

This is the part you need to understand conceptually.

When a request reaches:

/api/device-communication

the server basically performs:

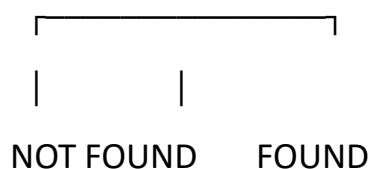
Receive request



Get device token



Check token in database



UNAUTHORIZED

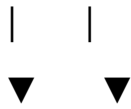


Too many requests?



YES NO

Check behavior



SUSPICIOUS AUTHORIZED

That's the heart of NETGUARD.

17. Tomorrow's actual demo flow

I would do it **exactly in this order**.

Before HOD arrives

Laptop:

```
cd path\to\NETGUARD
```

```
node server.js
```

Check that you see:

NETGUARD database connected.

...

NETGUARD server running on port 3000

Then open:

<http://localhost:3000>

Login.

Dashboard ready.

Then connect phone

Connect:

Laptop + Phone → same hotspot/network

Find laptop IP:

```
ipconfig
```

Look for:

IPv4 Address

For example:

10.24.89.40

Then phone:

<http://10.24.89.40:3000/device-test.html>

18. Then tell your story

Don't immediately start clicking randomly.

Start with the **problem**.

You can say:

"Our project started with a simple question: when a device communicates with a protected server, how can we determine whether that device should be trusted?"

Then:

"We identified two important cases. First, an unknown device should not be allowed to communicate. Second, an authorized device may still exhibit abnormal communication behavior."

Then:

"This led us to NETGUARD."

Now show dashboard.

19. Demo 1 — Normal

Enter your real phone token.

Click:

Send Network Request

Green result.

Say:

"This is our registered mobile device. NETGUARD recognizes its token and allows the communication."

Show dashboard.

20. Demo 2 — Unauthorized

Change token to:

FAKE-DEVICE-TOKEN

Click.

Red result.

Pause for a second.

Then say:

"Now I'm simulating a device that has not been registered with NETGUARD."

Show dashboard.

"The server cannot find this device token in the authorized-device database, so the request is blocked and a security alert is generated."

Very clean.

21. Demo 3 — Suspicious

Put the real token back.

Send requests rapidly.

When suspicious appears:

Pause.

Then say:

"Now I'm going to use the same authorized device, but generate an unusually high number of requests within a short period."

Then show orange alert.

"Notice the difference. This device is not unauthorized. It is a known device, but its behavior has become abnormal. NETGUARD therefore classifies it as suspicious rather than unauthorized."

That sentence is gold for your demo.

22. Your final conclusion

End with:

"So NETGUARD does not simply ask whether a device is known or unknown. It also considers how an authorized device behaves. Through this combination of device authorization and request-pattern monitoring, our system can identify unauthorized communication and abnormal network behavior."

Then:

"For our current prototype, we are monitoring communication with our protected server rather than attempting to inspect private traffic or intercept phone calls."

That last part keeps your technical claim accurate.

23. One thing you absolutely should NOT say

Don't say:

✗ "NETGUARD can detect every device on Wi-Fi."

Don't say:

✗ "NETGUARD can intercept phone calls."

Don't say:

✗ "NETGUARD monitors all traffic on the network."

Don't say:

✗ "It detects hackers."

Instead say:

✓ "NETGUARD monitors communication with our protected server."

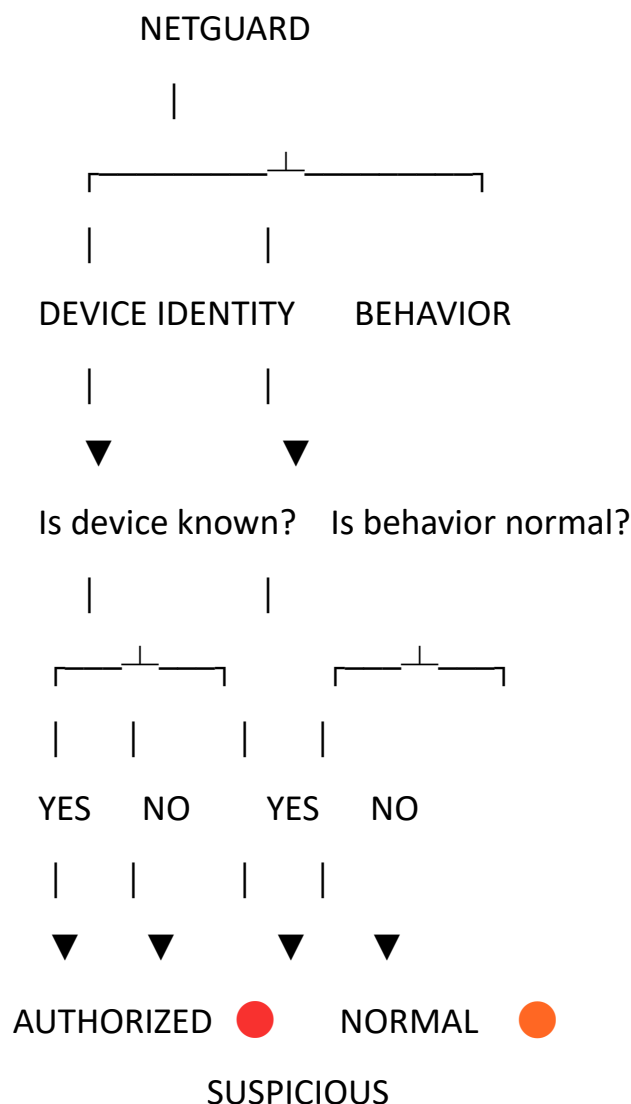
✓ "It identifies registered and unregistered devices attempting to communicate with the server."

✓ "It detects abnormal request patterns."

That's technically defensible.

24. And bro, you actually DO understand the project now

If you remember only this tomorrow, remember:



That's NETGUARD.

And tomorrow you don't need to rebuild anything. **Start the server, log in with your existing account, use the existing mobile token, connect the phone to the same network, and demonstrate the three states.**

Your code is already working. Tomorrow's job is mainly to **tell the story confidently and show the working system.**

When you tap **Wi-Fi → Guest Network**, the authentication of the Wi-Fi itself is normally handled by the **Wi-Fi access point/router and its authentication system**, not by a laptop running our current NETGUARD prototype.

So the real-world version would look like this:

Student's Phone



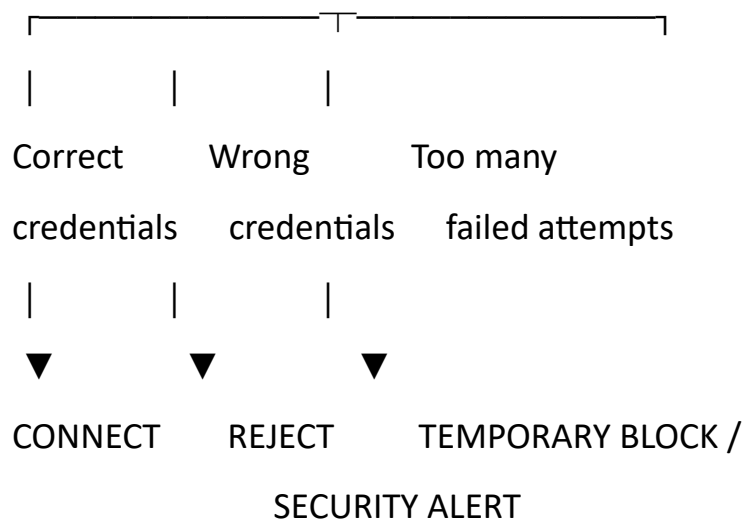
Selects "College Guest Wi-Fi"



Wi-Fi Access Point / Authentication Server



Checks credentials

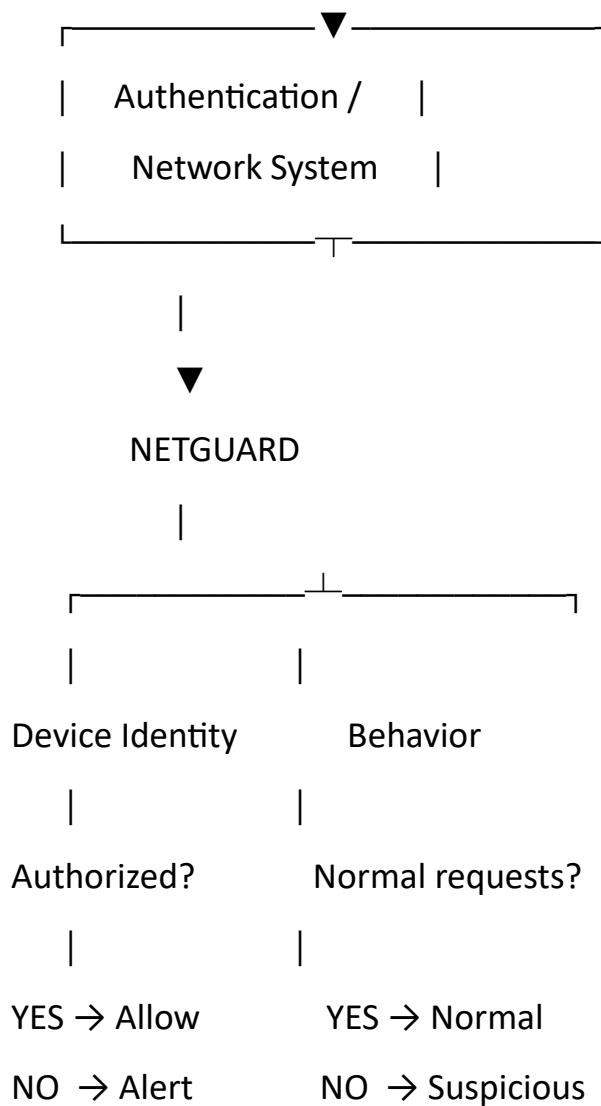


Where NETGUARD fits

Imagine the college has a security system integrated with the network infrastructure:

COLLEGE NETWORK





So suppose an MCA student connects to **MCA Wi-Fi**.

If their credentials/device are authorized:

● **Access allowed**

If an unknown device repeatedly tries to authenticate:

● **Unauthorized / security event**

If an already authorized device starts generating an unusual number of requests:

● **Suspicious behavior**

But don't say this tomorrow:

"NETGUARD checks the Wi-Fi password when I click the Wi-Fi button."

Because **our current prototype doesn't do that.**

Instead, say:

"In a real deployment, NETGUARD could be integrated with the institution's authentication and network infrastructure. The network's existing authentication system would handle Wi-Fi access, while NETGUARD would receive relevant security events and monitor authorized device communication and abnormal behavior."

That's actually a **much stronger explanation** because you're separating the **Wi-Fi authentication layer** from the **security monitoring layer**.

And your current phone demo is essentially a **small controlled simulation of the NETGUARD monitoring layer**:

REAL COLLEGE

OUR DEMO

College device	→ Your phone
Wi-Fi infrastructure	→ Laptop server
Authentication system	→ Device token
Security monitoring	→ NETGUARD
Unknown device	→ Fake token
Abnormal traffic	→ Rapid requests
Security dashboard	→ Your dashboard

So yes — **your college Guest/MCA network example is exactly the kind of real-world scenario you can use in your presentation**, just don't claim that our current laptop prototype itself controls the college Wi-Fi connection.

think of it like this

Your laptop is the **miniature laboratory version**.

Imagine your college has a huge security control room.

You can't bring the entire college network into your project tomorrow.

So you created:

A small controlled environment that demonstrates the same security concept.

Your laptop = small server.

Your phone = endpoint/device.

SQLite = small database.

NETGUARD dashboard = security administrator's screen.